Unit VI: Database Programming

6.1 Introduction to ADO.NET and ADO.NET Architecture

ADO.NET is a data access technology in .NET used for interacting with databases. It provides a bridge between applications and data sources, enabling data retrieval, manipulation, and updating.

ADO.NET Architecture

ADO.NET consists of:

- **Connection Object**: Establishes a connection to a database.
- **Command Object**: Executes SQL commands.
- **DataReader**: Reads data in a forward-only, read-only manner.
- **DataSet**: Stores data in an in-memory cache.
- **DataAdapter**: Acts as a bridge between DataSet and database.

6.2 DataReader, DataSet, DataTable, and DataAdapter

- **DataReader**: Provides fast, forward-only access to data.
- **DataSet**: A disconnected data container that can hold multiple tables.
- **DataTable**: Represents a single table in memory.
- **DataAdapter**: Fills a DataSet/DataTable and updates the database.

Example: Using DataReader

```
using System;
using System.Data.SqlClient;

class Program {
   static void Main() {
      string connectionString = "your_connection_string";
      using (SqlConnection con = new SqlConnection(connectionString)) {
         con.Open();
         SqlCommand cmd = new SqlCommand("SELECT * FROM Students", con);
         SqlDataReader reader = cmd.ExecuteReader();
         while (reader.Read()) {
            Console.WriteLine(reader["Name"]);
         }
      }
```

```
    }
}
```

6.3 Database Connection and Specific Classes

- **SqlConnection**: Manages the database connection.
- **SqlCommand**: Executes SQL commands.
- **SqlTransaction**: Manages transactions.

Example: Database Connection

```
using (SqlConnection con = new SqlConnection("your_connection_string")) {
    con.Open();
    Console.WriteLine("Database Connected");
}
```

6.4 Accessing Data with ADO.NET

CRUD Operations

- **Create** (Insert Data)

```
SqlCommand cmd = new SqlCommand("INSERT INTO Students(Name, Age)
VALUES ('John', 20)", con);
cmd.ExecuteNonQuery();
```

- **Read** (Retrieve Data)

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Students", con);
SqlDataReader reader = cmd.ExecuteReader();
```

- **Update Data**

```
SqlCommand cmd = new SqlCommand("UPDATE Students SET Age = 21 WHERE
Name = 'John'", con);
cmd.ExecuteNonQuery();
```

- **Delete Data**

```
SqlCommand cmd = new SqlCommand("DELETE FROM Students WHERE Name = 'John'", con);
cmd.ExecuteNonQuery();
```

Executing Commands

- **ExecuteNonQuery()**: Used for INSERT, UPDATE, DELETE commands.
- **ExecuteReader()**: Retrieves multiple rows.
- **ExecuteScalar()**: Returns a single value.

6.5 Stored Procedures

A stored procedure is a precompiled SQL script stored in the database.

Creating a Stored Procedure

```
CREATE PROCEDURE GetStudentName @id INT
AS
BEGIN
   SELECT Name FROM Students WHERE ID = @id
END
```

Executing Stored Procedure in C#

```
SqlCommand cmd = new SqlCommand("GetStudentName", con);
cmd.CommandType = CommandType.StoredProcedure;
cmd.Parameters.AddWithValue("@id", 1);
string name = cmd.ExecuteScalar().ToString();
Console.WriteLine(name);
```

6.6 Database and Entity Framework

Entity Framework (EF) is an ORM (Object-Relational Mapper) that simplifies database access.

Using EF to Retrieve Data

```
using (var context = new SchoolContext()) {
   var students = context.Students.ToList();
```

```
    foreach (var student in students) {
        Console.WriteLine(student.Name);
    }
}
```

Additional Topics

1. ADO.NET Connection Lifecycle

- **Open Connection**: con.Open();
- **Execute Command**: cmd.ExecuteNonQuery();
- **Retrieve Data**: cmd.ExecuteReader();
- **Close Connection**: con.Close();

2. Difference Between DataReader and DataSet

Feature	DataReader	DataSet
Connection Type	Connected	Disconnected
Data Fetching	Forward-only	Multiple tables
Performance	Fast	Moderate
Memory Usage	Low	High

3. Handling Transactions in ADO.NET

```
using (SqlTransaction transaction = con.BeginTransaction()) {
    try {
        SqlCommand cmd = new SqlCommand("INSERT INTO Students(Name, Age)
VALUES ('Mark', 22)", con, transaction);
        cmd.ExecuteNonQuery();
        transaction.Commit();
    } catch {
        transaction.Rollback();
    }
}
```